

When AI Agents Work Together

Understanding A2A, MCP, and the Future of Connected AI

Practical demonstrations with Microsoft Agent Framework

Udaiappa Ramachandran (Udai)

CTO|CSO @ Akumina Inc. | Microsoft MVP (AI)

Web: <https://udai.io>

LinkedIn: <https://linkedin.com/in/udair>

Meetup: <https://meetup.com/nashuaug>



Agenda

- The problem, and what A2A is (vs. MCP)
- Core concepts & how the pieces fit together
- How agents talk: transport & interaction patterns
- Maturity: what's production-ready vs. still evolving
- A2A in the Microsoft Agent Framework
- Why it matters: business value & enterprise use
- Live samples (.NET + Python), patterns & wrap-up

The Problem: Islands of Intelligence

- **Everyone is building agents** — every team, vendor, and framework ships its own
- **Each is a silo** — its own API, memory, tools, and prompts
- **Collaboration means glue code** — custom integration for every pairing
- **No common language** — no standard way to discover, call, or trust another agent



What is A2A?

An open standard for agent-to-agent communication

- **Framework- & vendor-neutral** — agents built by different teams work together
- **Open governance** — created by Google, donated to the Linux Foundation
- **Broad backing** — AWS, Cisco, Google, IBM, Microsoft, Salesforce, SAP, ServiceNow
- **Protocol milestone** — A2A reached v1.0, with SDKs available across Python, JavaScript, Java, C#/.NET, Go, and Rust

A2A vs. MCP — Complementary, Not Competing

MCP — agent to tool

- How an agent connects to APIs, data, and resources
- Equips a single agent with the capabilities it needs
- Think: the agent's hands

A2A — agent to agent

- How independent agents discover and delegate to each other
- Lets specialized agents collaborate across frameworks
- Think: the agent's phone

Use both together. A2A is not an agent framework, a tool-call protocol, or a chat app.

SDK Integration Is Not A2A

Via a vendor SDK

- Bedrock, Gemini, OpenAI, Microsoft Foundry, Copilot Studio
- Called via provider-specific SDKs or REST APIs
- Provider-specific authentication and request models
- May include proprietary discovery or orchestration
- Switching providers often requires integration changes

Via the A2A protocol

- Calls an A2A-compliant endpoint via a standard interface
- Agent Card describes skills, endpoints & security
- Standard messages, tasks, artifacts & operations
- Decouples the caller from the agent's framework
- Compliant agents are easier to add, replace, or compose

It is the protocol interface that makes an interaction A2A — not the model, SDK, framework, or provider. Some platforms now offer native A2A endpoints or adapters; others still need an A2A gateway or integration layer.

The Core Actors



User

- Human or automated service
- Defines the goal or request
- Starts the interaction



A2A Client

- The client (calling) agent
- Acts on the user's behalf
- Initiates the A2A call

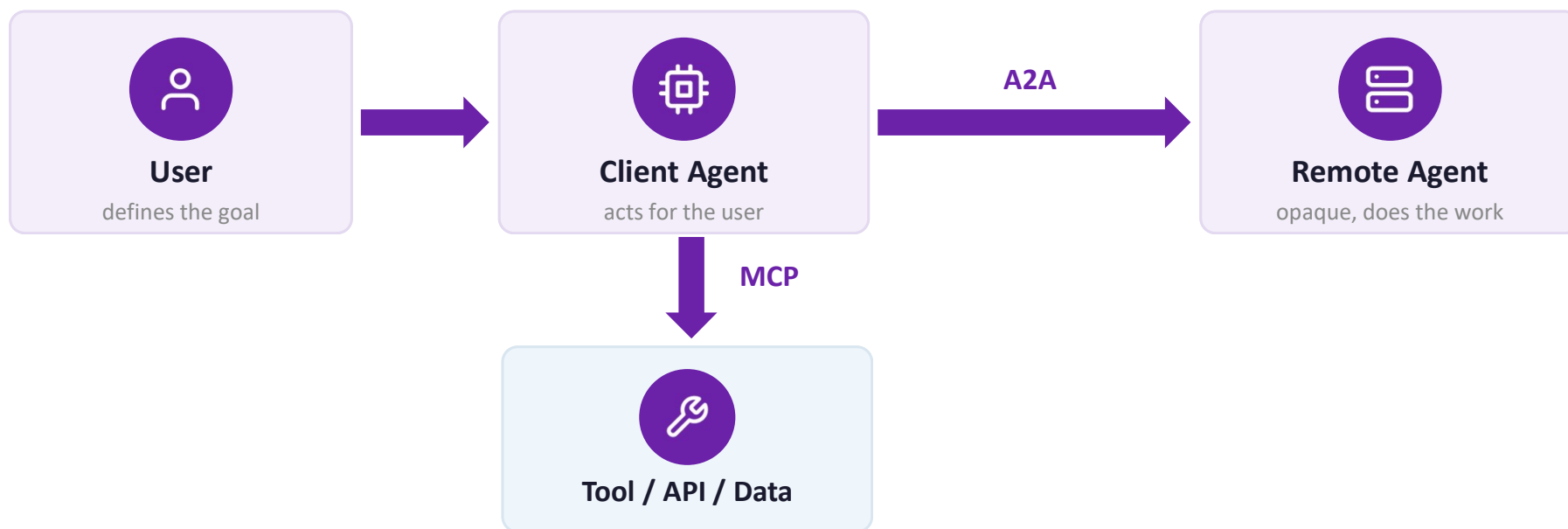


A2A Server

- The remote agent
- Exposes an HTTP endpoint
- Opaque: internals stay private

"Opaque" is the trust unlock — collaborate without exposing your IP or internal state.

How A2A Works — The Flow



A2A enables direct, standardized collaboration between agents. MCP connects an agent to its tools. The two compose.

The Vocabulary of A2A

- **Agent Card** — JSON "business card": identity, endpoint, skills, auth requirements
- **Task** — a stateful unit of work with an ID and lifecycle (long-running jobs)
- **Message** — a single turn of conversation, role of user or agent
- **Part** — the content inside a message: text, file, or structured data
- **Artifact** — a concrete deliverable produced during a task (doc, image, data)

Agent Cards & Discovery

How agents find and size each other up

- **Published as JSON** — every A2A Server makes an Agent Card available
- **Declares capabilities** — name, URL, streaming support, skills, auth
- **Card-first flow** — clients read the card, then decide fit and how to call
- **Well-known path** — a standard discovery location; registries & direct config also supported

How Agents Talk

A2A 1.0 defines JSON-RPC, gRPC, and HTTP+JSON bindings (secured via standard transport-level auth)

Result-delivery patterns:

Request / Response

- Send a request, get a reply
- Poll for long-running jobs
- Simplest pattern

Streaming (SSE)

- Open HTTP connection
- Real-time incremental updates
- Great for live UX

Push Notifications

- Server calls your webhook
- For very long / disconnected work
- No open connection needed

Life of a Request: Message vs. Task

Message

- Returned when the agent can answer immediately
- A single turn of communication
- Think: quick Q&A

Task

- Returned for long-running work
- Has an ID and a trackable lifecycle
- Artifacts are returned or updated as work progresses

contextId groups related messages and tasks into one conversation.

A2A Maturity — Stable Core, Evolving Tooling



Stable in A2A 1.0

- Core operations and data model
- Agent Cards
- Tasks, messages, parts, and artifacts
- JSON-RPC, gRPC, and HTTP+JSON bindings
- Streaming, polling, and push-notification patterns



Implementation maturity varies

- SDK and framework package stability
- Provider-native endpoint availability
- Multi-tenancy and identity propagation
- Operational tooling and interoperability testing
- Framework-specific adapters

Build on the stable A2A 1.0 core; implementation and package maturity still vary by SDK and provider.

Where the Microsoft Agent Framework Fits

- **Direct successor** — to Semantic Kernel & AutoGen for agent development and orchestration
- **Multi-language** — .NET and Python, with Go in public preview
- **Expose your agents** — publish them so any A2A client can call
- **Consume remote agents** — call external A2A agents as if they were local
- **First-class A2A** — a core integration, alongside MCP, AG-UI, and M365

Why Should I Care?



Lower integration cost

Stop writing custom glue for every agent pairing — connect once, via a standard.



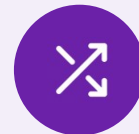
Faster delivery

Compose specialist agents on demand instead of rebuilding capabilities in-house.



Enterprise security alignment

Standard auth patterns; internal prompts, tools & memory stay private. Data disclosure still follows app policy.



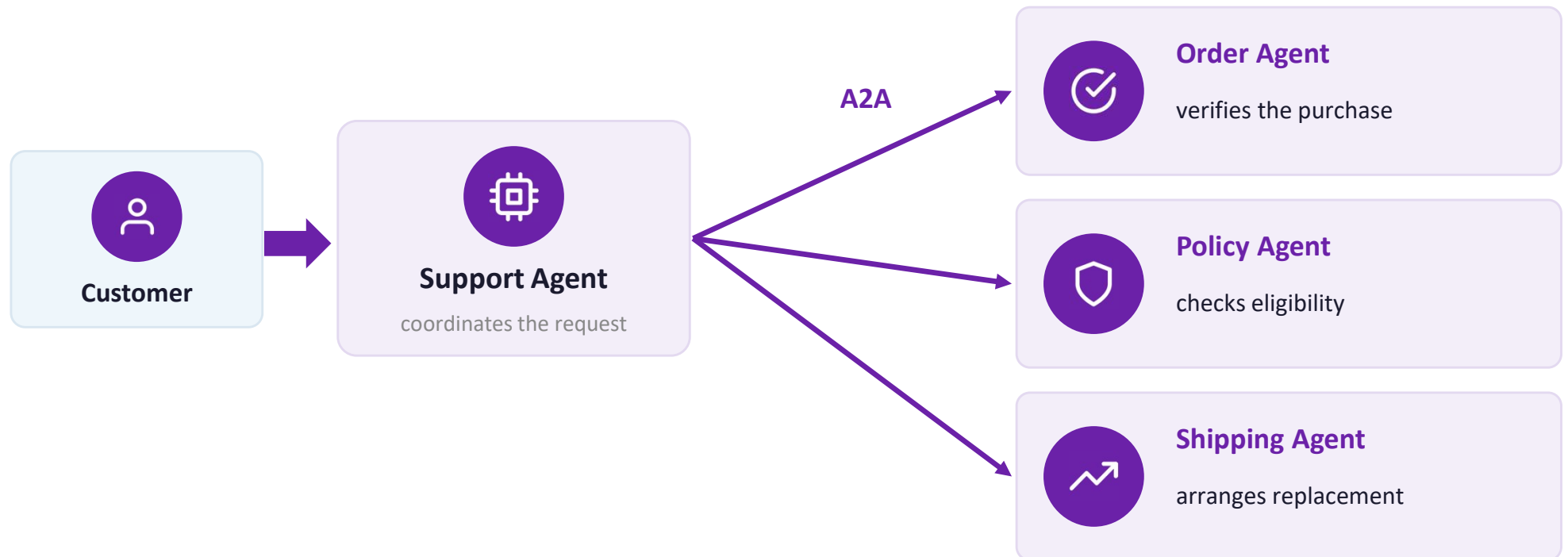
Vendor flexibility

Compose best-of-breed agents across teams and vendors with less integration code and lock-in.

Multi-vendor workflows, specialist teams, partner integrations — without sharing their internal implementation.

One Request, Many Agents

A customer wants to replace a damaged product — the support agent delegates the work over A2A.



Each agent is built and managed independently — they collaborate only through A2A. Swap the shipping agent for a compatible partner agent with minimal changes to the rest of the workflow.

Sample 1 — Expose an Agent (.NET)

MapA2A publishes an agent in a few lines

```
var agent = builder.AddAIAgent(  
    "pirate", instructions: "Speak like a pirate.");  
  
app.MapA2A(agent, "/a2a/pirate", agentCard: new()  
{  
    Name = "Pirate Agent", Version = "1.0"  
});
```

*One mapping call exposes the configured agent through an A2A endpoint.
Full sample → github.com/microsoft/agent-framework*

Sample 1 — Calling It (the A2A Request)

Simplified request flow — a standards-compliant message in, message out

```
POST /a2a/pirate/v1/message:stream
{
  "message": {
    "role": "user",
    "parts": [{ "kind": "text", "text": "..."}],
    "contextId": "foo"
  }
}
```

Reuse contextId to continue a conversation. GET .../v1/card returns the Agent Card.

Sample 2 — Consume a Remote Agent (Python)

A2AAgent wraps any remote endpoint as a local agent

```
resolver = A2ACardResolver(httpx_client=client, base_url=host)
card = await resolver.get_agent_card()

async with A2AAgent(agent_card=card, url=host) as agent:
    response = await agent.run("What are your capabilities?")
```

*pip install agent-framework-a2a • the SDK handles all protocol details.
Full sample on [GitHub](#).*

Sample 2 — Streaming & Long-Running Work

Real-time updates and background tasks

```
# Stream tokens in real time (SSE)
stream = agent.run("...", stream=True)
async for update in stream: ...

# Long jobs → continuation token to poll later
resp = await agent.run("...", background=True)
result = await agent.poll_task(resp.continuation_token)
```

Streaming = live, Message-style UX. background=True = Task lifecycle.

Sample 3 — Serve Your Own Agent (Python)

A2AExecutor exposes an Agent Framework agent over A2A

```
request_handler = DefaultRequestHandler(  
    agent_executor=A2AExecutor(agent),  
    task_store=InMemoryTaskStore(),  
    agent_card=public_agent_card)
```

*A2AExecutor turns your agent into a discoverable A2A server.
The mirror image of Sample 2 — full sample on [GitHub](#).*

Architecture & Patterns

Delegation

- A host agent routes sub-tasks to specialized remote agents

Agents as Tools

- Wrap remote A2A agents as callable tools of a host agent

Cross-Framework Mesh

- Mix Agent Framework, LangGraph, CrewAI, and custom agents

MCP + A2A Together

- Each agent uses MCP for tools, A2A to collaborate

Resources & Q&A

A2A Protocol & Spec

a2a-protocol.org

Agent Framework Repo

github.com/microsoft/agent-framework

A2A SDKs

github.com/a2aproject

A2A SDKs

github.com/nhcloud/a2a

Agent Framework A2A Docs

learn.microsoft.com/agent-framework/integrations/a2a

Python A2A Samples

[github.com/microsoft/agent-framework \(python/samples/04-hosting/a2a\)](https://github.com/microsoft/agent-framework/tree/main/python/samples/04-hosting/a2a)

A2A Samples

github.com/a2aproject/a2a-samples

Start small: expose one agent, or wrap one remote agent, today.

THE TAKEAWAY

A2A lets independent agents collaborate without exposing their internal implementation.

Thank you! Let's connect and build interoperable agents.

Udai — udai.io | linkedin.com/in/udair